DAW_M03_T07_ACT7_CristianCantero

Este ejercicio es un proyecto de POO para la gestión de un gimnasio. El objetivo es comprender clases, objetos, relaciones, arrays, diseño de sistemas y patrones.

El flujo de construcción ha sido el siguiente.
1- construcción de las clases
2- Métodos de las clases. Construcción y modelado.
3- Construcción del menú en la clase Main.

<u>El objetivo</u> es que una clase sea el "cerebro" de operaciones utilizando objetos instanciados con sus atritutos y sus métodos de las otras clases. Finalmente será la clase main la encargada de llamar a estos métodos a través de un menú que interactuará con el usuario.

Patrones:
- array + contador + método añadir
- Búsqueda utilizando bucle for
- Relación bidireccional (composición de clases)
- Creación + inserción

<u>Clases</u>

Cliente: Responsable de tener sus datos personales y entrenador asignado.
Entrenador: Responsable de tener sus datos personales y lista de clientes.
Gimnasio: Responsable de gestionar arrays de clientes y entrenadores.
Main: Responsable de llamar a todos los métodos para ejecutar el programa.

<u>CONSTRUCCIÓN DE LAS CLASES</u>

CLIENTE
Esta clase es la responsable de gestionar sus datos internos. Empieza declarando sus atributos con modificador de acceso private (encapsulación poo). Sus atributos serán: nombre, dni, edad, telefono, email, membresia y un objeto de la clase Entrenador (necesario para el patrón de relación bidireccional). Inicializamos un constructor vacío y otro con todos sus atributos. Después aplicamos todos los getters y setters. En mi caso ctrl + i puedo insertar los constructores y los getters y setters.

ENTRENADOR
Esta clase es responsable de gestionar sus datos internos exactamente igual que la clase Cliente. Pero, además, también tiene que utilizar un método con patron de array+contador. De esta manera podremos asignar entrenador al cliente (parte fundamental del ejercicio).
Los atributos seran nombre, codigoEntrenador, edad, telefono, email, especialidad, experiencia (años), un array de tipo Cliente de 10 posiciones y el numDeClientesAsignados para saber el número de clientes que tiene.

Detalles importantes:

- El array es de 10 posiciones porque asi lo manda el ejercicio. Cada entrenador solo puede tener 10 clientes.
- Las variables numDeClientesAsignados y clientesAsignados se me colaron en un principio como *falsos parámetros.* Un falso parámetro es aquel que se recibe en el constructor pero no se utiliza, ya que su valor no debe depender de datos externos, sino del propio estado interno del objeto. En este caso estas variables representan la estructura interna del entrenador. ClientesAsignados es el array donde se almacenan los clientes y numDeClientesAsignados es el contador que indica cuantos clientes hay realmente, y nos sirve para que quede guardado el siguiente hueco en el array. Estos atributos no se reciben desde fuera, sino que los define el propio objeto al crearse.

Esta clase también le aplico los getters y setters a fin de tener disponible estos métodos para modelar el programa.

GIMNASIO

Esta clase es el 'cerebro' del programa. Se encarga de gestionar los arrays y métodos de cliente y entrenador. También es la responsable de tener las funciones o métodos que vamos a utlizar para gestionar el programa.
Empezamos con el patron *array* + *contador* + *método añadir*

```
/*
patrón:
    array + contador + método añadir
 */
private Cliente[] clientes = new Cliente[100];
private Entrenador[] entrenadores = new Entrenador[10];  //==> array
private int numClientes;  //==> contador
private int numEntrenadores;
```

Declarar los arrays que son objetos de Cliente y Entrenador y despues los contadores numClientes y numEntrenadores. Los contadores nos sirven de herramienta para guardar los datos. No solo sirven para saber el número de datos guardados, sino también nos indica cual es el siguiente hueco en el array.

Constructor vacío y vamos con los métodos.

agregarCliente(Cliente nuevoCli)

```
public void agregarCliente(Cliente nuevoCli) {
    if (numClientes < clientes.length) {
        clientes[numClientes] = nuevoCli;
        numClientes++;
    } else {
        System.out.println("Aforo completo");
    }
}
```

Este método va a guardar el cliente dentro del array clientes.
Por eso recibe como parámetro un objeto cliente llamado nuevoCli.
Primero realizamos la comprobación a través del condicional. Si el array tiene huecos disponibles guarda el cliente y le suma uno al contador. De esta manera llevamos la cuenta del número de clientes y sabemos cual es el siguiente hueco dentro del array. Si el array estuviera lleno nuestro else imprime un mensaje de aforo completo.

crearCliente(Scanner sc)

```java
public void crearCliente(Scanner sc) {
    Cliente c = new Cliente();

    System.out.println("Nombre: ");
    String nombre = sc.nextLine();
    c.setNombre(nombre);
    System.out.println("Dni: ");
    String dni = sc.nextLine();
    c.setDni(dni);
    System.out.println("Edad: ");
    int edad = Integer.parseInt(sc.nextLine());
    c.setEdad(edad);
    System.out.println("E-mail: ");
    String email = sc.nextLine();
    c.setEmail(email);
    System.out.println("Teléfono: ");
    String telefono = sc.nextLine();
    c.setTelefono(telefono);
    System.out.println("Membresía (1. premium/ 2. básico: ");

    int membresia = Integer.parseInt(sc.nextLine());
    if (membresia != 1 && membresia != 2) {
        System.out.println("Usa 1 o 2");
    }
    c.setMembresia(membresia);
    agregarCliente(c);
}
```

Este método se encarga de recoger los datos del cliente usando un patron de creación+inicialización+inserción. Es decir: Crea el objeto, rellena con datos y lo inserta en el sistema.
Pasos:
1. Creacion. Aquí nace el objeto. *Cliente c = new Cliente();*
2. Inicialización. Se rellena con datos. Ejemplo: c.setNombre(nombre);
3. Inserción ==> agregarCliente(c);
Para la inserción utilizo el método agregarCliente que agarra el objeto con sus datos y lo guarda en el array.

Detalle importante ==> La membresía la vamos a manejar de la siguiente forma: Con un condicional if le indicamos que si el usuario no introduce 1 o 2 le salta un mensaje de error.

De la misma forma que hemos hecho con el Cliente. Es decir, un método que recoje la información del objeto y otro método que lo inserta en el array. Modelaremos los métodos del Entrenador. Tendremos un método *agregarEntrenador()* que insertará en el array los objetos entrenador y otro método *crearEntrenador()* que se encargara de la creación + inicialización + inserción.

```
    public void agregarEntrenador(Entrenador nuevoEnt) {
        if (numEntrenadores < entrenadores.length) {
            entrenadores[numEntrenadores] = nuevoEnt;
            numEntrenadores++;
        } else {
            System.out.println("Aforo completo");
        }
    }

    public void crearEntrenador(Scanner sc){
            Entrenador e = new Entrenador();
            System.out.println("Nombre: "); String nombre = sc.nextLine();
            e.setNombre(nombre);
            System.out.println("Código de entrenador: " ); int codigoEntrenador = sc.nextInt(); sc.nextLine();
            e.setCodigoEntrenador(codigoEntrenador);
            System.out.println("E-mail: "); String email = sc.nextLine();
            e.setEmail(email);
            System.out.println("Edad: "); int edad = sc.nextInt(); sc.nextLine();
            e.setEdad(edad);
            System.out.println("Especialidad: "); String especialidad = sc.nextLine();
            e.setEspecialidad(especialidad);
            System.out.println("Años de experiencia: "); int experiencia = sc.nextInt(); sc.nextLine();
            e.setExperiencia(experiencia);
            System.out.println("Teléfono: "); int telefono = sc.nextInt(); sc.nextLine();
            e.setTelefono(telefono);
            agregarEntrenador(e);
    }
```

agregarEntrenador() Recibe como parámetros un objeto entrenador. Comprueba a través del contador que el array tiene huecos. Si es asi lo guarda y sube el contador mas uno.

crearEntrenador() Recibe como parametro un objeto Scanner para que el usuario pueda interactuar.

Pasos:
1. Creacion. Aquí nace el objeto. *Entrenador e = new Entrenador();*
2. Inicialización. Se rellena con datos. Ejemplo: c.setNombre(nombre);
3. Inserción ==> agregarEntrenador(e);

Para la inserción utilizo el método agregarEntrenador() que agarra el objeto con sus datos y lo guarda en el array.

asignarEntrenador()

```java
public void asignarEntrenador(String dniCliente, int codigoEntrenador) {
    Cliente clienteEncontrado = null;
    Entrenador entrenadorEncontrado = null;

    for (int i = 0; i < numClientes; i++) {
        if (clientes[i].getDni().equalsIgnoreCase(dniCliente)) {
            clienteEncontrado = clientes[i];
            break;
        }
    }

    for (int i = 0; i < numEntrenadores; i++) {
        if (entrenadores[i].getCodigoEntrenador() == (codigoEntrenador)) {
            entrenadorEncontrado = entrenadores[i];
            break;
        }
    }

    if (clienteEncontrado == null) {
        System.out.println("No se encontró cliente");
        return;
    }
    if (entrenadorEncontrado == null) {
        System.out.println("No se encontró entrenador");
        return;
    }
    //Asignación
    clienteEncontrado.setEntrenador(entrenadorEncontrado);
    entrenadorEncontrado.agregarNuevoCliente(clienteEncontrado);
    //Patrón: Relación bidireccional
}
```

Recibe como parámetros un String dniCliente y un int codigoEntrenador. Datos necesarios para asignar un entrenador a un cliente.
El método empieza declarando dos variables que hacen referencia a dos objetos. Un objeto Cliente y un objeto Entrenador. No pueden inicializarse a 0 porque no son valores primitivos, son referencias a obetos. Se inicializan a **null** porque en ese momento no estan apuntando a ningun objeto en memoria. Es decir, todavía no se ha encontrado ningún cliente ni entrenador.

Cliente clienteEncontrado = null;
Entrenador entrenadorEncontrado = null;

Durante la ejecución de los bucles for, estas variables cambiarán su valor si se encuentra una coincidencia. Si al final de la búsqueda no se encuentran clientes o entrenadores las variables seguiran siendo null. Lo cual permite controlar la situación mediante un condicional.

```
if(clienteEncontrado == null) {
        sout("No se encontró ningún cliente")
        }
```

Pareciere que estas variables son cajas que guardarán el contenido para luego usarlo.

Ahora vamos con el for que se encarga de guardar los datos dentro de esas variables.

El for que encuentra la coincidencia dada por la inserción del usuario, utiliza el numClientes como contador para iterar. Si el dni que hay dentro de clientes* (getDni()) coincide con el dniCliente introducido por el usuario, clienteEncontrado lo guarda.

¿Donde esta clientes?¿Que es clientes? Clientes no es una variable local del método, sino un atributo de la clase.

Private Cliente[] clientes = new Cliente[100]; … (l. 11)

Este array actúa como la estructura donde se almacenan todos los objetos Cliente. Aquí es donde guardamos los clientes. Si no usamos este array, el método no tiene ningún sentido.

Ahora voy con el **for** en su plenitud. *Vamos allá*. (línea 89)

El bucle for recorre el array clientes usando el contador numClientes.
El índice **[i]** (l.90) apunta a una posición o hueco dentro de **clientes**. En el hueco donde el puntero está en ese momento, en esa iteración. Podemos, gracias a los getters y setters, acceder a la información del objeto para usarla (get) o cambiarla (set). Entonces, dentro del hueco o posición busco el dni (getDni). Lo paso por un filtro que ignora las mayusculas y lo pone todo en minúsculas (equalsIgnoreCase) y lo comparo con el dato introducido por el cliente (dniCliente). Si coinciden **lo guardo**.

¿Como lo guardo?

clienteEncontrado=clientes[i]; (l.91)

Acordemonos de esto=> *Cliente clienteEncontrado = null; (l.86)*

clienteEncontrado ya no está vacío. Es ahí donde guardamos el dato que nos ha proporcionado el puntero que nos tenía situado en ese hueco o posción. Ese puntero no es otro que **clientes[i]**. El consiguiente break; nos saca del bucle.

Ahora vamos con el que es (para mi) uno de los puntos álgidos. La asignación del cliente al entrenador. Para esto vamos a utilizar el patrón de realción bidireccional.

(L.111)
//Asignación
clienteEncontrado.setEntrenador(entrenadorEncontrado);
entrenadorEncontrado.agregarNuevoCliente(clienteEncontrado);
//Patrón: Relación bidireccional

En la primera línea se establece que el cliente tiene asignado ese entrenador. Podriamos decir que el cliente 'conoce' a su entrenador.
En la segunda línea se añade al cliente a la lista de clientes del entrenador.
Una relación bidireccional es aquella en la que dos objetos se conocen mútuamente. Implica que ambos objetos se actualizan mutuamente para reflejar la misma relación. Ahora el cliente sabe quién es su entrenador y el entrenador sabe qué clientes tiene.

Nota personal: Cuando estaba realizando esta asignación no era capaz de actualizar los dos objetos. Me faltaban piezas. Por esta razón tuve que crear el método *agregarNuevoCliente(Cliente c)* en la clase Entrenador y usarla en la clase Gimnasio dentro de este método. A esta situación se le dice => **encapsulación + responsabilidad de clase.**

*entrenadorEncontrado.**agregarNuevoCliente**(clienteEncontrado);*

agregarNuevoCliente() se encuentra dentro de la clase Entrenador.

```
public void agregarNuevoCliente(Cliente c){
    if(numDeClientesAsignados<clientesAsignados.length){
        clientesAsignados[numDeClientesAsignados]=c;
        numDeClientesAsignados ++;
    }else{
        System.out.println("Aforo completo");
    }
}
```

Responde exáctamente con la misma lógica de agregarCliente() o agregarEntrenador().

mostrarInfoClientes()

```java
public void mostrarInfoClientes() {
    System.out.println("--Listado de clientes--");
    for (int i = 0; i < numClientes; i++) {
        Cliente c = clientes[i]; // acostumbrate a mirar 'parriba'

        System.out.println();
        System.out.println("Nombre: " + c.getNombre());
        System.out.println("Dni:" + c.getDni());
        System.out.println("Edad:" + c.getEdad());
        System.out.println("Email:" + c.getEmail());
        System.out.println("Teléfono:" + c.getTelefono());
        System.out.println("Membresía: " + c.getMembresia());
        if (c.getMembresia() == 1) {
            System.out.println("Membresia Básica");
        } else {
            System.out.println("Membresia Premium");
        }

        if (c.getEntrenador() != null) {
            System.out.println("Entrenador: " + c.getEntrenador().getNombre());
        } else {
            System.out.println("Este cliente no tiene entrenador asignado");
        }
    }
}
```

Este método no recibe parámetros. Se va a encargar de entrar en el array, iterar uno a uno las posiciones donde estan guardados los objetos y mostrarnos su info uno por uno.
Para recorrer el array de Clientes vamos a usar un bucle for. Dentro del bucle declaramos un objeto Cliente para conectarlo con el ínidice del bucle. Y recorremos mostrando la información. Cuando llegamos a la membresía hay que aplicar lógica para saber si es básica o premium. Si cuando accedemos al nombre del entrenador no encontramos ningún problema el programa nos proporcionará el nombre. Sino, nos mostrará un mensaje de que no hay entrenador asignado.

mostrarInfoEntrenadores()

```java
public void mostrarInfoEntrenadores() {
    System.out.println("--Listado Entrenadores--");
    for (int i = 0; i < numEntrenadores; i++) {
        Entrenador e = entrenadores[i];
        Cliente[] lista = e.getClientesAsignados();

        System.out.println();
        System.out.println("Nombre: " + e.getNombre());
        System.out.println("Código de entrenador: " + e.getCodigoEntrenador());
        System.out.println("Edad: " + e.getEdad());
        System.out.println("Especialidad: " + e.getEspecialidad());
        System.out.println("Experiencia: " + e.getExperiencia() + " años.");
        System.out.println("Teléfono: " + e.getTelefono());

        if (e.getNumDeClientesAsignados() == 0) {
            System.out.println("Este entrenador no tiene clientes asignados");
        } else {
            System.out.println("--Clientes de " + e.getNombre() + " --");
            for (int j = 0; j < e.getNumDeClientesAsignados(); j++) {
                Cliente c = lista[j];
                System.out.println("----------------------------------");
                System.out.println("Nombre del cliente: " + c.getNombre());
                System.out.println("Dni del cliente: " + c.getDni());
                System.out.println();

            }
        }//patron: estructura de datos anidada
    }
}
```

Este método tiene una lógica muy parecida al método anterior pero se diferencia por el tipo de patrón. En este caso vamos a usar un **patron de estructura de datos anidada.** Le decimos así porque nos encontramos con un bucle **for** dentro de otro **for**.

El primer bucle for usará el contador numEntrenadores para iterar dentro del array de entrenadores. Declaramos un objeto entrenador que contendrá el índice con el que vamos a recorrer el bucle. Y declaramos tambiíen un objeto Cliente que tendrá una lista con los clientes asignados (e.getClientesAsignados)

La iteración para mostrar info es como las anteriores. Usaremos el metodo get para acceder a la info y mostrarla. Pero para manejar los clientes asignados necesitaremos un poco más de lógica. Con un condicional miraremos si el numDeClientesAsignados es igual a 0. Si fuera asi el entrenador no tendria clientes. Si no fuera asi el sistema muestra los clientes del entrenador.

Bloque Main

Esta clase es la que ejecuta el programa.

```java
public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Gimnasio g = new Gimnasio();

        int opcion = 0;
        System.out.println("--Welcome to GymBros--");
        System.out.println();

        do {
            System.out.println("Selecciona una opción: ");
            System.out.println("1. Agregar cliente");
            System.out.println("2. Agregar entrenador");
            System.out.println("3. Asignar entrenador a cliente");
            System.out.println("4. Mostrar clientes");
            System.out.println("5. Mostrar entrenadores");
            System.out.println("0. Salir");
            opcion = Integer.parseInt(sc.nextLine());

            switch (opcion) {
                case 1:
                    g.crearCliente(sc);
                    break;
                case 2:
                    g.crearEntrenador(sc);
                    break;
                case 3:
                    System.out.println("Introduce el dni del cliente: ");
                    String dniCliente = sc.nextLine();
                    System.out.println("Introduce el código del entrenador");
                    int codigoEntrenador = sc.nextInt();
                    sc.nextLine();

                    g.asignarEntrenador(dniCliente, codigoEntrenador);
                    break;
                case 4:
                    g.mostrarInfoClientes();

                    break;
                case 5:
                    g.mostrarInfoEntrenadores();

                    break;
                case 0:
                    System.out.println("Programa cerrado correctamente.");
                    return;
                default:
                    System.out.println("Introduce una opción válida.");
            }

        } while (opcion != 0);

    }
}
```

Este bloque declara un objeto Scanner con elque el usuario introducira los datos. Declaramos un objeto Gimnasio para conectar la clase con el main. Utilizaremos para el menú una estructura de switch/case envuelta en un try/catch y a su vez envuelta en un do/while. Los case van llamando a los métodos sin más hasta llegar al case 3. Donde se pide el dni, el código del entrenador y se le pasan al método asignarEntrenador() para que este haga su trabajo.

g.asignarEntrenador(dniCliente, codigoEntrenenador);
- De la clase Gimnasio agarra el método asignarEntrenador() y utiliza el dniCliente y codigoEntrenador que te acabo de pasar via usuario. -

Si no me equivoco, en este tramo estamos usando *encapsulación* y *separación de responsabilidades*. Lo primero porque sin saber que hay dentro de asignarEntrenador() lo estamos utilizando solo pasandole dos argumentos. Y lo segundo porque la función no se encarga de pedir los datos del usuario, sino que lo hace el main.

En el case 0 puse programa cerrado correctamente y ya me imagino que hubiera sido mas “correcto” ponerlo fuera del case. Pero me funcionaba y lo dejé de esta forma.

BLOQUE DEL OUTPUT.

En este bloque voy a poner un ejemplo del funcionamiento de mi programa.

```
run:
--Welcome to GymBros--

Selecciona una opción:
1. Agregar cliente
2. Agregar entrenador
3. Asignar entrenador a cliente
4. Mostrar clientes
5. Mostrar entrenadores
0. Salir
```

Se abre el menú con las opciones

```
1
Nombre:
Cristian
Dni:
44567332F
Edad:
49
E-mail:
cristian.cantero77@gmail.com
Teléfono:
+34 776898959
Membresía (1. premium/ 2. básico:
1
Selecciona una opción:
1. Agregar cliente
2. Agregar entrenador
3. Asignar entrenador a cliente
4. Mostrar clientes
5. Mostrar entrenadores
0. Salir
```

Introduce bien los datos a la hora de agregar un cliente.

```
Selecciona una opción:
1. Agregar cliente
2. Agregar entrenador
3. Asignar entrenador a cliente
4. Mostrar clientes
5. Mostrar entrenadores
0. Salir
2
Nombre:
Peter
Código de entrenador:
1
E-mail:
peter@petr.com
Edad:
32
Especialidad:
cross-fit
Años de experiencia:
5
Teléfono:
+34 456789823
```

Seleccionamos la opción 2. E introducimos bien los datos del entrenador.

Y cuando utilizamos la opción 3 … el programa me dejó ingresar los datos, pero a la hora de validar… explotó. *Quiero dejar constancia en el trabajo porque* *<u>para mi esto es una guia</u>*. Asi que vamos a resolver este entuerto.

```
Selecciona una opción:
1. Agregar cliente
2. Agregar entrenador
3. Asignar entrenador a cliente
4. Mostrar clientes
5. Mostrar entrenadores
0. Salir
3
Introduce el dni del cliente:
44567332F
Introduce el código del entrenador
1
Exception in thread "main" java.lang.NullPointerException: Cannot read the array length because "this.clientesAsignados" is null
        at Act07.Entrenador.agregarNuevoCliente(Entrenador.java:32)
        at Act07.Gimnasio.asignarEntrenador(Gimnasio.java:113)
        at Act07.Main.main(Main.java:45)
/Users/cristiancantero/Library/Caches/NetBeans/26/executor-snippets/run.xml:111: The following error occurred while executing this line:
/Users/cristiancantero/Library/Caches/NetBeans/26/executor-snippets/run.xml:68: Java returned: 1
BUILD FAILED (total time: 18 minutes 15 seconds)
```

Primero de todo el error. Java nos dice que this.clientesAsignados es null. Lo primero que hago es mirar de donde viene este this.
this.clientesAsignados = new Cliente[10]; es la línea 26 de la clase Entrenador.
Básicamente la traducción del error es *'el array no esta inicializado'*

Y ese es el problema, el array no está inicializado porque el constructor no lo tiene dentro.

```
13        private int numDeClientesAsignados;
14
15        public Entrenador() {
16        }
17
```

Este constructor vacío no tenía inicializados ni clientesAsignados ni numDeClientesAsignados.

Por lo tanto cambiamos el código

```
14
15        public Entrenador() { //
16            this.clientesAsignados = new Cliente[10];
17            this.numDeClientesAsignados = 0;
18        }
```

Ahora ya están inicializados.

Y volvemos a la carga con las pruebas…

```
Selecciona una opción:
1. Agregar cliente
2. Agregar entrenador
3. Asignar entrenador a cliente
4. Mostrar clientes
5. Mostrar entrenadores
0. Salir
4
--Listado de clientes--

Nombre: Cristian
Dni:44567332F
Edad:49
Email:cristian.cantero77@gmai.com
Teléfono:+34 334453245
Membresía: 1
Membresia Básica
Este cliente no tiene entrenador asignado
```

Opción 4. Listado de clientes.

```
Selecciona una opción:
1. Agregar cliente
2. Agregar entrenador
3. Asignar entrenador a cliente
4. Mostrar clientes
5. Mostrar entrenadores
0. Salir
5
--Listado Entrenadores--

Nombre: Peter
Código de entrenador: 1
Edad: 32
Especialidad: cross-fit
Experiencia: 5 años.
Teléfono: 34
Este entrenador no tiene clientes asignados
```

Opción 5. Lista de entrenadores.

```
Selecciona una opción:
1. Agregar cliente
2. Agregar entrenador
3. Asignar entrenador a cliente
4. Mostrar clientes
5. Mostrar entrenadores
0. Salir
3
Introduce el dni del cliente:
44567332F
Introduce el código del entrenador
1
Selecciona una opción:
1. Agregar cliente
2. Agregar entrenador
3. Asignar entrenador a cliente
4. Mostrar clientes
5. Mostrar entrenadores
0. Salir
4
--Listado de clientes--

Nombre: Cristian
Dni:44567332F
Edad:49
Email:cristian.cantero77@gmai.com
Teléfono:+34 334453245
Membresía: 1
Membresia Básica
Entrenador: Peter
```

Y ahora la opción 3 asignamos entrenador al cliente.

<==Escogemos opción 3

<==Escogemos opción 4

<==Peter aparece como entrendador

```
--Listado Entrenadores--

Nombre: Peter
Código de entrenador: 1
Edad: 32
Especialidad: cross-fit
Experiencia: 5 años.
Teléfono: 34
--Clientes de Peter --
-----------------------------------
Nombre del cliente: Cristian
Dni del cliente: 44567332F
```

Y finalmente el cliente figura en el listado de entrenadores.

<===Datos del cliente.

CONCLUSIONES FINALES:

He puesto mucho cariño y dedicación a la hora de documentar este ejercicio. Habrán sido 12 horas tratando de entender todo lo que sucede y como sucede. Mi intención principal (a parte de aprobar el trabajo por supuesto) es que me sirva de guia para estudiar. En este trabajo aparecen muchísimas cosas que estoy seguro que son 'patron' a la hora de trabajar con el desarrollo de software. De todos modos la conclusión final es la de siempre: **Hay que estudiar mucho.**

BIBLIOGRAFÍA CONSULTADA:

Documentación: clases Linkia, apuntes Linkia.
Herramientas de aprendizaje: NoteBook LM, Chat GPT.